Introduction

Every so often, LeetCode gives you a problem that looks exactly like one you've already solved — until you read the constraints closely and realize the whole strategy you'd reach for by muscle memory is wrong. Minimum Moves to Equal Array Elements III is one of those problems. It sits right next to "Minimum Moves to Equal Array Elements II" in name and in spirit, and interviewers love pairs like this because they test whether you actually understood the previous solution or just memorized "sort, then median."

If you've been through enough array problems, your instinct probably says: sort the array, pick the median, sum the distances. That instinct is correct for a huge family of problems — but it's built on an assumption you need to question every single time: can you move values in both directions? The moment a problem takes that freedom away and only lets you move one way, the median stops being optimal, and you need a different mental model entirely.

This problem is worth sitting with because it isn't really about arrays at all. It's about recognizing constraints. The array, the "add one" operation, the one-directional move — all of that is scaffolding. What the problem is actually testing is whether you can notice when a familiar pattern doesn't apply and derive the right one from first principles. That skill shows up constantly in interviews, in system design, and honestly in day-to-day engineering: knowing when your mental shortcuts don't fit the situation in front of you.

Problem Overview

You're given an array of integers. In a single move, you may pick any one element and increase it by exactly 1 — that's the only operation allowed. You cannot decrease any value. Your job is to find the minimum total number of moves required to make every element in the array equal.

The constraints keep things small (at most 100 elements, with modest value sizes), which is a strong hint that the intended solution doesn't need anything fancy like sorting or binary search — it's signaling that a single linear pass is enough.

The key phrase to underline is "increase it by exactly 1." There's no mention of decreasing. That one-way restriction is the entire problem. Everything else — finding gaps, summing distances — follows directly from it.

Example

Take the array `[2, 1, 3]`.

- The largest value in the array is `3`.
- To raise `2` up to `3`, you need `3 - 2 = 1` move.
- To raise `1` up to `3`, you need `3 - 1 = 2` moves.
- The `3` itself needs `3 - 3 = 0` moves, since it's already at the target.

Total moves: `1 + 2 + 0 = 3`.

Why is the target `3` and not something smaller, like `2`? Because `3` can never come down to `2` — the only move available is `+1`. Any target below the current maximum is unreachable for at least one element. So `3`, the maximum, is the only value every element can actually reach.

Intuition

Here's where experience either helps you or actively misleads you. A lot of "equalize the array" problems reward meeting in the middle — pick the median (or average, for a looser version), and let higher values come down while lower values go up. That minimizes total movement because every element is allowed to travel toward the center from either side.

Try applying that instinct to `[2, 1, 3]`. The values average out to about `2`. But look at what that would require: the `3` would need to drop to `2`. That's a decrease — and decreases aren't on the menu. The averaging trick doesn't just give a suboptimal answer here; it gives an *impossible* one.

This is the moment to stop and ask a sharper question: given that elements can only go up, what's the smallest value that every element in the array can legally reach? It can't be anything below the current maximum, because the maximum element would have to shrink to get there, and shrinking isn't allowed. It also doesn't need to be anything above the maximum, because that would mean extra, unnecessary moves for every element, including the one that's already at the top.

That leaves exactly one candidate: the maximum value already sitting in the array. It's the one number that requires zero moves for itself, and it's the lowest common target every other element can actually climb to. Once you see that, the rest of the problem collapses into arithmetic — add up how far every element is from that one fixed point.

Brute Force Solution

Idea: Try every possible target value one at a time, from the smallest element up to some reasonable ceiling, compute the total moves needed for each candidate target, and keep the minimum.

Algorithm: 1. For each candidate target `t` from `min(nums)` up to `max(nums)`: - Check that `t >= x` for every `x` in `nums` (otherwise some element would need to decrease). - If valid, compute `sum(t - x for x in nums)`. 2. Track the smallest total across all valid targets.

Advantages: It's a direct translation of "try everything," so it's easy to reason about and hard to get wrong on the logic front.

Disadvantages: It's doing a huge amount of redundant work. Every target below the maximum fails the validity check instantly, and every target above the maximum only adds extra moves. The brute force never actually discovers anything the direct approach doesn't already tell you — it just confirms it the slow way.

Time complexity: O(n · k), where `k` is the range of candidate targets you test (potentially as large as the value range in the array). **Space complexity:** O(1) beyond the input, assuming you don't store intermediate sums.

Optimal Solution

The optimal approach skips the search entirely because we already proved the target can only be one value: the maximum.

Step	What happens	Cost
1	Scan the array once to find `mx = max(nums)`	O(n)
2	Scan the array again, summing `mx - x` for every `x`	O(n)
3	Return the total	O(1)

There's no sorting, no median, no candidate search. The two passes can even be merged into one if you want to shave constant factors, but keeping them separate keeps the code obvious — find the ceiling, then measure every element's distance from it.

It helps to picture the earlier "glasses of water" framing: you can only pour water in, never out. The fastest way to make every glass the same height is to fill each one up to whichever glass is already fullest. You never average the glasses, because averaging would ask the fullest glass to lose water — the one thing you're not allowed to do.

Python Code

```
class Solution:
    def minMoves(self, nums: list[int]) -> int:
        mx = max(nums)
        return sum(mx - x for x in nums)
```

Code Walkthrough

- `class Solution:` — the required wrapper LeetCode expects your method inside.
- `def minMoves(self, nums: list[int]) -> int:` — takes the array of integers, returns a single integer: the minimum move count.
- `mx = max(nums)` — one linear pass to find the largest value in the array. This is the fixed target every element must reach, for the reasons worked out in the Intuition section.
- `return sum(mx - x for x in nums)` — a second linear pass. For every element `x`, `mx - x` is exactly the number of `+1` moves needed to bring it up to the target. Since `mx` is the maximum, this value is always non-negative, so you never accidentally compute a "negative move count" for an element that's already past the target — nothing can be past it.
- The generator expression inside `sum()` avoids building an intermediate list, keeping the extra space at O(1).

Dry Run

Using `nums = [2, 1, 3]`:

1. `mx = max([2, 1, 3])` → `mx = 3`.
2. Compute each gap:
3. `3 - 2 = 1`
4. `3 - 1 = 2`
5. `3 - 3 = 0`
6. Sum: `1 + 2 + 0 = 3`.
7. Return `3`.

Matches the example worked through earlier: raising `2` to `3` costs one move, raising `1` to `3` costs two moves, and `3` needs nothing.

Complexity Analysis

Time: O(n). Finding the maximum requires inspecting every element once. Summing the gaps requires inspecting every element once more. Two linear passes are still linear overall — O(n) + O(n) = O(n).

Space: O(1). Only `mx` and a running sum are kept in memory; nothing scales with the input size. (Python's `max()` and the generator inside `sum()` don't materialize extra arrays.)

Both bounds are tight — you cannot do better than O(n), because you must at minimum look at every element once to even know what the maximum is.

Alternative Solutions

You could merge the two passes into a single loop that tracks a running maximum and a running sum simultaneously, then correct the sum at the end by accounting for how the maximum changed mid-scan. It's a fun exercise, but it adds bookkeeping for no asymptotic benefit — you're still O(n) time and O(1) space, just with messier code. The two-pass version is more readable and just as fast in practice, which is the better trade for interview settings.

Sorting the array first is a trap worth naming explicitly, since it's the move this problem is designed to tempt you into. Once sorted, the maximum is just the last element — but sorting costs O(n log n), strictly worse than the O(n) you get by calling `max()` directly. Sorting is only useful here if you also need the array in order for some other reason; for this problem alone, it's pure overhead.

Edge Cases

- **Empty array:** Not reachable under LeetCode's stated constraints (the array always has at least one element), but if it somehow occurred, `max()` on an empty sequence raises a `ValueError` — worth knowing even if you don't need to handle it here.
- **Single-element array:** `max(nums)` is that element itself, every gap is `0`, and the answer is `0`. No moves needed since there's nothing to equalize against.
- **All elements already equal:** Every gap is `0`, so the total is `0` regardless of array length.
- **Duplicate maximums:** If the largest value appears more than once, each occurrence contributes a gap of `0` — it doesn't change the logic or double-count anything.
- **Maximum value at the array's boundary constraints:** Even at the largest allowed values, Python's integers don't overflow, so there's no special handling needed for large sums.

Common Mistakes

1. **Reaching for the median or average**, carried over from the "move in both directions" version of this problem. That approach produces a target smaller elements can't reach without decreasing, which this problem forbids.
2. **Sorting the array unnecessarily.** It doesn't break correctness, but it adds O(n log n) overhead for a problem that's solvable in O(n).
3. **Computing `x - mx` instead of `mx - x`**, which silently flips every gap negative and corrupts the total.
4. **Assuming the target should be the minimum**, confusing "add" with "decrease toward the smallest value" — the direction of the allowed move determines the target, not an arbitrary choice of which extreme to use.
5. **Recomputing `max(nums)` inside the summing loop** (e.g., calling `max()` on every iteration instead of once up front), which quietly turns an O(n) solution into O(n^2).

Interview Questions

- What changes about your approach if moves could go either direction (raise or lower)?
- How would you handle this if the array were a stream and you couldn't store the whole thing in memory?
- Can you prove that the maximum is the unique optimal target, rather than just asserting it?
- What's the time complexity if you were asked to also return *which* elements needed the most moves?
- How would your solution change if each move could add any positive integer, not just 1?

Similar Problems

- **Minimum Moves to Equal Array Elements II** — the two-directional sibling of this problem, where the median replaces the maximum as the optimal target. Comparing the two is the best way to internalize how a constraint on move direction changes the entire strategy.
- **Minimum Moves to Equal Array Elements** — the original version, equivalent in spirit to the "raise to the max" problem, often phrased with increment/decrement moves that net out the same way.
- **Candy** — another problem where a one-directional constraint (each child needs more candy than specific neighbors) forces a non-obvious target rather than a simple average.
- **Minimum Number of Operations to Make Array Continuous** — shares the theme of figuring out what the "right" target range is before computing a cost over it.

Key Takeaways

- The moment a problem restricts movement to one direction, the median and the average stop being valid targets — don't apply them on autopilot.
- When only increases are allowed, the maximum is the sole value every element can legally reach, and it costs the fewest total moves by construction.
- Small constraints in a problem statement are often a hint about the intended time complexity — here, they're telling you a single pass (or two) is all you need.
- The real skill being tested isn't the final one-liner; it's recognizing when a familiar pattern from a similar-sounding problem doesn't transfer.

Watch the Video

If you'd rather see this derived step by step — including the glasses-of-water framing and a couple of quick quiz checks along the way — watch the full walkthrough here: {LINK}

About the Series

This breakdown is part of **Fun with Learning Technology**, a series built around making coding interview prep actually stick — not by memorizing solutions, but by building the intuition that lets you derive them on the spot. Each video takes one problem, questions the obvious approach, and walks through why the real solution works the way it does.

Call To Action

If this walkthrough helped the problem click, give the video a like on YouTube and subscribe for more of these breakdowns — new ones go up regularly. If you'd rather get the written version straight to your inbox, subscribe to the Substack newsletter. And if you spotted a different way to think about this problem, drop it in the comments — these threads often turn into some of the best explanations in the series. Sharing this with someone prepping for interviews helps the channel grow, too.

Quick Review

Pattern: array equalized via +1/-1 moves. What's the tell?

When each move only shifts one element by 1 and you need min total moves, think median minimization, not average.

Time complexity to solve this problem?

O(n log n), dominated by sorting the array before computing median and gap sums.

Space complexity of the median-based solution?

O(1) extra if sorting in place, O(n) if a sorted copy is made.

Common bug when summing distances to the target?

Forgetting to pick the actual median (not average) causes undercounting moves, especially with even-length arrays.

Average vs median: why does average fail here?

Average minimizes squared distance; moves cost is absolute distance, so median (not mean) minimizes total steps.

Interview follow-up: array has duplicates, does approach change?

No — median logic and gap-summing still hold since duplicates don't affect position-based minimization.
